

Idea Paper

FactBank: Teaching Untrained Models the Libraries They Never Saw

Paper I: The Idea and Its Use: concept, motivation, audience, and real-world application

By Mohannad. This paper was drafted by an AI assistant, because I do not yet know how to write a formal research paper. The FactBank project, its ideas, decisions, and solutions, are mine; the AI helped with the coding, the brainstorming, and with writing this paper. It is grounded only in the project's own files, and where something is unknown it is marked as such.

1. In one sentence

FactBank makes a stock large language model answer correctly about fast-moving software libraries it was never trained on, by carrying a small, searchable bank of **“landmine facts”** inside the model’s own file, so the model keeps doing the reasoning while the bank supplies the exact, current knowledge it is missing. There is no fine-tuning, no retraining, and no external retrieval service: the retrieval runs inside the model at inference time.

The thesis, proven: knowledge and reasoning are separable. Hand a model the exact fact it is missing at the moment it needs it, and even a 2-billion-parameter edge model answers like it was trained yesterday, on libraries it never saw. (README.md)

2. The problem it addresses

The knowledge inside a language model’s weights is frozen at its training cutoff, but the libraries people write code against are not. Between cutoff and deployment, a fast-moving library ships releases that **rename attributes, remove functions, flip default arguments, or change a return type with no error at the call site**. A model that “knows” the library from training will confidently emit the old API. The failure is quiet: the code looks right, the imports resolve, and it breaks only at runtime, or worse, silently computes the wrong thing.

The project names the dangerous facts precisely. A **landmine fact** is one of exactly three kinds:

- **post-cutoff**, the change shipped after the model’s training cutoff, so the model cannot have learned it;
- **reverses a trained habit**, a rename, a moved import, or a flipped default that the model still “knows” the old way;
- **silent failure**, wrong-but-runs behaviour, where there is no error message for the model to self-correct against.

These are the facts that bite. They are the difference between a demo that works and code that ships a subtle bug. FactBank targets them and nothing else, it does not carry a library’s

documentation, only the handful of sentences per library where the current version disagrees with the model's memory.

The same idea reaches one step further, into **security**. An insecure-but-working call, loading a model file without `weights_only=True`, parsing XML that resolves external entities, using `random` where `secrets` is required, is just another kind of landmine: it runs, it looks right, and it is quietly wrong in a way no error will flag. A model trained on years of insecure example code has learned the insecure habit as its default. FactBank can carry the **secure** default as a landmine fact and hand it over at exactly the moment the model is about to write the risky line, so the model writes secure code by default without being asked to.

3. The core idea

The insight is that **reasoning was never the missing piece**. The coding and reasoning ability comes from the model itself, and that part is already strong even at small sizes. What smaller models lack is not intelligence, it is up-to-date knowledge; they were simply never trained on the newest library versions. FactBank fills exactly that gap: it hands the model the current facts right when it needs them, so the model can apply the skills it already has.

The system is built around three deliberately separated parts:

- **The fuzzy core**, the model's weights. It holds reasoning and code structure and is never asked to recall an exact post-cutoff fact.
- **The frozen bank**, a curated, read-only file of landmine facts, each tagged with a library, a version, and a verbatim source quote.
- **The code-driven loop**, plain code (not a model decision) that retrieves the relevant facts and places them in front of the model before it answers.

Because reasoning is not the bottleneck, a small model plus the bank closes much of the distance to a big model on the libraries the bank covers. The project's own framing of the size curve is telling: **small models need the bank most, and big models use it best**.

3.1 How it differs from the usual answers

The conventional response to stale model knowledge is either to **fine-tune / retrain** the model on new material, or to bolt on **retrieval-augmented generation (RAG)**, a live document store queried per request. FactBank takes a third path: distil the correction down to landmine facts, and ship it **inside the model artifact itself**. Compared with the alternatives, the concept-level differences are:

Property	Fine-tune / retrain	Document RAG	FactBank
Touches the weights?	Yes	No	No, metadata only
Needs a live service?	No	Yes (vector store / retriever)	No, runs in the chat-template

What is carried	New parameters	Raw documents	Curated one-line landmine facts
Who decides to retrieve	n/a	Model tool-choice or pipeline	Deterministic code, no tool-choice
Swap knowledge later	Retrain	Re-index store	Re-bake / edit the JSONL bank
Portability	Model-specific	Server + model	One GGUF file, runs anywhere

The consequence is portability: a stock GGUF model file, loaded anywhere with no companion server, answers version-correctly. The knowledge “stays yours” in an editable file you own rather than in a service or a set of frozen weights.

4. Who it is for

The idea is aimed squarely at people who run models locally or at the edge, and who care about a specific, fast-moving domain:

- **Local and edge deployers**, anyone running a 2B-30B model on their own hardware, offline or air-gapped, who cannot call a frontier cloud model but still needs current-version correctness.
- **Security and networking practitioners**, the shipped experts (network security, offensive security / reverse engineering, the eBPF data plane, and a broad application-security expert) target exactly the tooling and the secure-coding habits this audience needs, where tool renames, version churn, and quietly-insecure defaults are constant.
- **Privacy- and cost-sensitive teams**, the whole correction lives in a file, so no data leaves the machine at inference and there is no per-query retrieval bill.
- **Domain builders and plugin authors**, anyone who wants to turn a generic model into an “expert” on a chosen stack can build a bank from that stack’s migration guides in an afternoon for roughly four cents of API cost.

The audience is explicitly not people who need general reasoning improvements or new algorithmic capability, FactBank supplies knowledge, not reasoning, and is honest that it cannot do the latter.

5. Real-world use cases

The project now ships a line of **four domain experts** on Hugging Face, each baked into three sizes (a ~2B edge model, a 12B, and a 26B mixture-of-experts). Each is a stock Gemma model with a domain bank baked into its chat-template; you run it with llama.cpp and the bank fires in-engine. The three original experts cover network security, offensive security / reverse engineering, and the eBPF data plane; the measured gains on their own landmine questions are large:

Expert (domain)	Libraries / facts	2B base→baked	12B base→baked	26B base→baked
netsec, security & networking	7 libs / 114 facts	39.6 → 66.7%	56.2 → 81.2%	77.1 → 93.8%
offsec, offensive security / RE	17 libs / 489 facts	27.3 → 84.1%	27.3 → 97.7%	38.6 → 97.7%
dataplane, eBPF / data plane	7 libs / 318 facts	10.6 → 68.1%	34.0 → 87.2%	42.6 → 91.5%

These figures are hand-verified (a human read the real model outputs; the automated scorer was used only to decide what to read). They cover libraries such as cryptography, OpenSSL 3, paramiko 3, urllib3 2, angr, pwntools, impacket, frida, netexec (formerly CrackMapExec), libbpf, Cilium, DPDK, and VPP, exactly the fast-churning tooling where a model's training is most likely stale.

5.1 The flagship: a model that writes secure code by default

The fourth and largest expert is an **application-security** expert, and it is the clearest demonstration of the idea reaching beyond version churn. Instead of a short list of libraries, its bank is organised by **weakness**, a large collection of **insecure-by-default landmines**: the places where the plausible, trained-in way to write a line of code is the insecure way. Carry the secure default as a fact, fire it at the right moment, and the model writes safe code without being asked. Concrete examples the bank handles: loading a model file with `torch.load(weights_only=True)` (a post-cutoff safe default), parsing XML with external entities turned off, choosing `yaml.safe_load` over `yaml.load`, secrets over random, parameterised SQL over string-built queries, reading credentials from the environment instead of hardcoding them, and constant-time comparisons for secrets.

Two things make this expert notable in plain terms:

- **It fires on innocent-sounding prompts.** The bank is indexed by the **coding task** (“load this checkpoint”, “parse this config”), not by the name of the vulnerability, so it helps even when the user never says the word “security”. That is the whole point: developers ask for a feature, not for a CWE.
- **It measurably beats a cloud model at the larger sizes.** On an external, third-party security benchmark (SecurityEval), on the common machine-scoreable subset, the 12B-and-bank and 26B-and-bank experts write more secure code than a much larger cloud model (DeepSeek-V4) run on the same prompts; the ~2B edge expert lands roughly level with it. The bank helps most where the base model is weakest and on specific weaknesses (unsafe deserialization, XML external entities, weak randomness, hardcoded credentials). **To keep the comparison fair, the caveats are stated plainly:** the cloud model was run in its ordinary, no-reasoning mode to match these editions (its reasoning mode might score higher); the subset is small (21 of 121 tasks, the ones a checker could judge) and **hand-scored**; DeepSeek-V4 is a strong cloud model but not the very largest frontier system; and only the

12B/26B experts beat it, while the 2B only draws level. The honest reading is plain: a bigger model is more secure to begin with, and the bank adds a real, repeatable lift on top.

The application-security bank is broad, roughly four thousand facts drawn from seven permissive, well-known security sources (MITRE's CWE, GitHub's CodeQL, the Bandit and gosec scanners, OWASP's mobile testing guide, RustSec, and standard crypto/networking guidance), spanning more than ten programming languages and many frameworks, with about a quarter of the facts carrying a verbatim insecure-versus-secure code pair. Every fact was checked twice: first that its quote is real, then a second adversarial pass that read the facts for correctness and removed the small fraction (~3.8%) that were wrong or out of date, because a quote being **real** does not make it **still true**.

5.2 Two editions: a fast one and one that shows its reasoning

Each expert ships in two editions that differ only in one setting:

- a **fast, reliable default** ("thinking-off") that answers directly and is the recommended everyday choice, and
- a **"shows-its-reasoning" edition** ("thinking-on") that thinks out loud before answering.

The reason for two is a subtle, honest finding the project surfaces: when a model is allowed to reason freely, it can talk itself **out** of a correct landmine fact and back into its old habit. The fast edition avoids that entirely; the reasoning edition keeps the trace but presents the fact as authoritative so the model trusts it. Even so, on the very hardest reasoning prompts the thinking-on edition can occasionally return a blank answer instead of code, which is inconvenient but **fails safe**, a blank is never insecure code. Most users should simply run the fast edition; the reasoning edition is there for anyone who wants to see the model's working.

5.3 More experts on the way, and where the pattern applies

Beyond the shipped four, the repository has built (but not yet baked) banks for further departments, an **AI-ML** expert (817 facts across OpenAI, LangChain, LlamaIndex, transformers, vLLM, pydantic, and more), a **Web full-stack** expert (Tailwind v4, React 19, Svelte 5, Vue 3, SQLAlchemy 2.0, Next.js 15, and others), plus DevOps, Databases and Web3 banks, illustrating the pattern: any domain with fast release cadence, habit reversals, quietly-insecure defaults, and a real reference document is a candidate expert.

Illustrative situations

- A penetration tester running a local model on an isolated network asks it to script against netexec; without the bank the model reaches for the retired CrackMapExec API, with the bank it uses the current one.
- An engineer asks for "code to load this saved model," never mentioning security, and the application-security expert quietly writes the `safe weights_only=True` version instead of the insecure default the model was trained on.

- An engineer generating code against pandas 3.0 or NumPy 2 gets the removed/renamed API right (e.g. `np.float_` removed → `np.float64`) rather than plausible-but-broken code.
- A team standing up an offline coding assistant bakes their own stack's migration guides into the model so it stops recommending deprecated calls.

6. How someone actually uses it

6.1 The easy path, download and run

For the published experts there is nothing to build. Download the GGUF from Hugging Face and serve it with llama.cpp; the bank is already inside the file:

```
llama-server -m gemma-4-12B-offsec-expert-Q4_0.gguf --jinja --port 8080
```

The `--jinja` flag is what makes the baked chat-template do the retrieval. From there, any OpenAI-compatible client points at the local endpoint. The prompt is identical to a stock model's, the only difference is whether the bank fires, which is also what makes a base-versus-expert comparison clean.

One practical note for the largest bank: the application-security template is big enough to exceed a size limit that some desktop loaders enforce on model metadata, so that expert also ships through an LM Studio Hub route that hands the full template to the engine directly. The published Hugging Face repository for each size carries both editions and the settings needed to run them.

6.2 The builder path, make your own expert in an afternoon

The project's central playbook (BUILD-AN-EXPERT.md) takes a stock base model to a running expert in seven stages. The cost is about four cents of extraction API and an afternoon of work:

- **Pick a library and find its migration guide.** The best targets combine fast version churn, habit reversals, and a real prose upgrade doc. This step is called "the one law": source targeting decides everything, a perfect extractor on the wrong document yields a bank of the wrong facts.
- **Extract the facts with one command.** An LLM reads the guide and fills a strict fact schema; dumb code verifies every field; a verbatim source quote is the anti-hallucination gate.
- **Assemble the bank,** place the verified facts, keep the bank rich (landmine-only, but do not drop facts just because a model might already know them), and sanity-check that every fact loads and ids are unique.
- **Adapt the bank for baking,** map each fact into the text the model reads plus a searchable side-channel of task words.
- **Bake the bank into the GGUF,** splice an inverted-index retriever plus the facts into the model's chat-template. The weights are untouched; it is a metadata rewrite.
- **Serve the expert** with llama-server and the model's native sampling settings.

- **Prove it works**, write landmine questions, run base versus expert on identical settings, and score by hand.

There is also a second consumption mode for people who prefer a service over a baked file: an installable **factbank** package that wraps any local OpenAI-compatible model in a sealed “draft → retrieve → refine” loop over an editable JSONL bank, and serves the result as one OpenAI-compatible endpoint. The model stays stock; the knowledge stays in a file you can edit.

A point of accuracy about delivery routes. The **current, shipping way** to put a bank inside a model is the inverted-index bake described above (the method behind all the published models). The project also contains an older “**static bake**” that simply dumps every fact into a fixed system prompt with no retrieval, that route is **legacy and superseded**, dormant and never used to ship any of the published models. Separately, a pure-Python **test bench** is used to validate facts and the retrieval logic offline; it is a development and validation harness, not a way to ship a model. So a builder’s real choice is between the shipping inverted-index bake (knowledge inside the file) and the served-loop package (knowledge in an editable file beside a stock model).

It is worth being honest about one gap between those two routes, because it explains a difference the numbers show. The served-loop prototype can reach a higher ceiling than the shipped file, because it retrieves not only on the user’s prompt but also on the model’s own first draft, which often names the risky call a benign prompt never mentions. That extra “draft key” is the single biggest lever for security prompts, and it cannot run inside a baked chat-template. So where this paper cites a served-loop result, it is describing the retrieval *method’s* ceiling, not the shipped model, and the shipped-model security numbers above are the more conservative, in-file ones.

7. What to expect, and what not to

Two habits are stressed throughout the project and are worth internalising before use. First, **score by hand**: automated scoring is used only to decide what to read; the verdict is a human reading the outputs, because the automated count has misled in both directions. Second, **verify currency, not just grounding**: the verbatim quote proves a fact is real, not that it is still true, a documented change can later be reverted, and a stale “fact” corrects the model to the wrong answer, which is worse than no fact at all.

FactBank corrects version-API knowledge, renames, removals, default flips, silent behaviour changes, and insecure-by-default habits that a reliable source documents. It does **not** supply reasoning, algorithms, or numerical methods; a problem that needs a numerically stable implementation rather than a documentable API change is outside what any fact bank can fix. And there is a subtle catch the project surfaces honestly: letting the model think before answering can make it argue an injected habit-reversal fact away and revert to its training. That is exactly why each expert ships in two editions, the fast “thinking-off” default that sidesteps the

problem, and a “thinking-on” edition that keeps the fact authoritative so the model trusts it over its prior, at the cost of an occasional blank (never-insecure) answer on the hardest prompts. The Technical and Verdict papers examine that mechanism and its limits in depth.

The honest summary the project itself offers: **the bank supplies knowledge, the model supplies reasoning, and the two must be told which one wins when they disagree.**